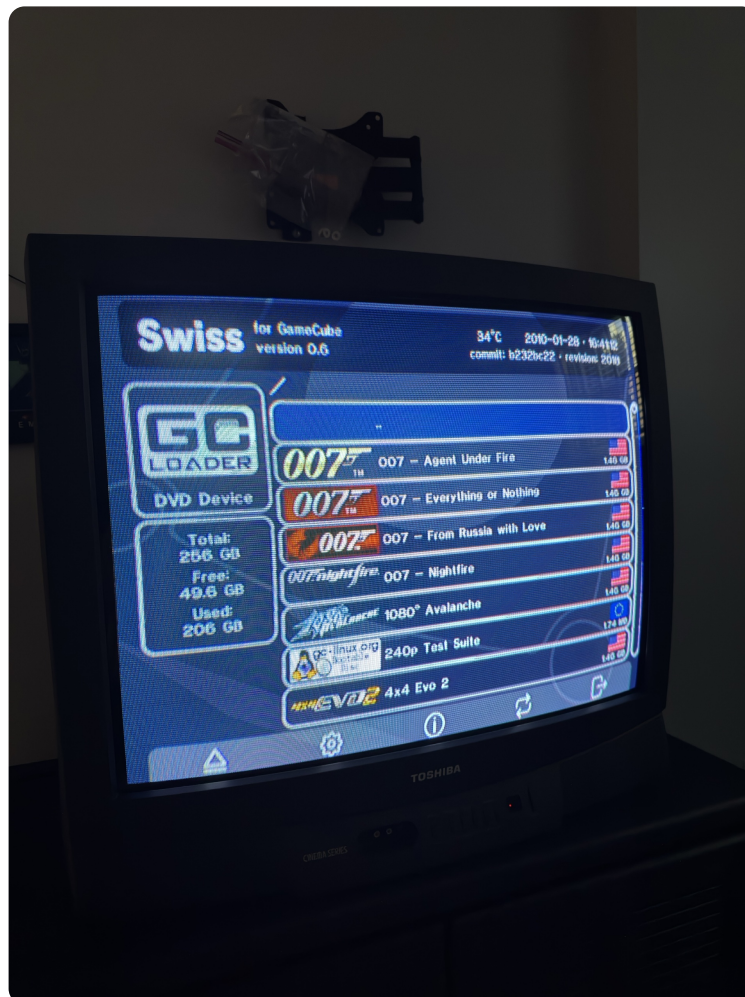


Carby Component: GCVideo 3.1 Flashing Guide

How to back up the firmware on an Insurrection Industries Carby Component cable and replace it with GCVideo 3.1, using an FT232H USB board.



October 6, 2026

Proceed at your own risk.

What you need

Item	Note
CJMCU FT232H board (or any FT232H breakout)	The USB board that talks to the cable's flash chip. Pins AD0 to AD4 and GND are used.
Six female-to-female jumper wires	Five go between the FT232H and the cable's J1 header. The sixth (from AD4) is touched to FPGA pin 100 by hand.
A thin probe wire or needle	For touching FPGA pin 100. The pins are very close together, so a fine tip helps.
A computer running Linux, or Windows with WSL2 and usbipd-win	WSL2 runs Linux inside Windows. usbipd-win passes the FT232H from Windows to WSL2.
Python 3 with pyftdi	pyftdi is the Python library the flashing scripts use. Setup is on page 4.
The GameCube and the cable	The console powers the cable, so it stays on while you flash.
A TV with component input	To check the result.

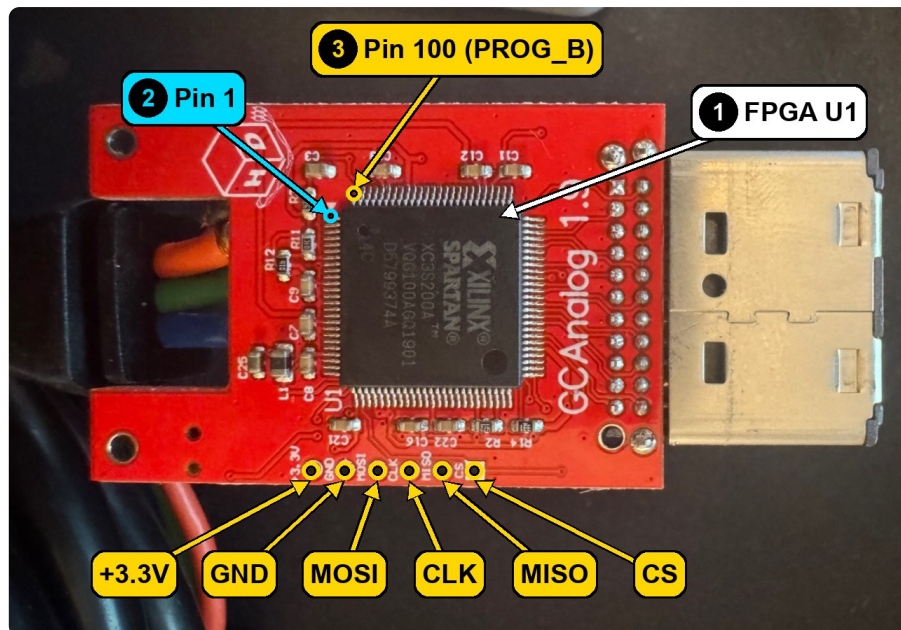
Files from this repository

- `firmware/gcvideo/carby_gcvideo31_full.bin`: the GCVideo 3.1 firmware.
- `firmware/original/carby_backup.bin`: the original firmware, for going back.
- `src/flasher/`: the scripts `dump.py`, `flashfull.py` and `restore.py`.

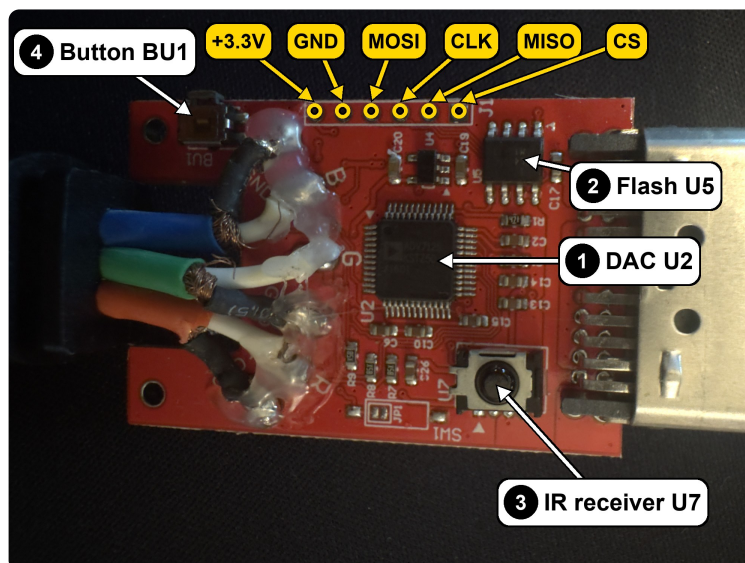
Before you start: this means opening the cable's plug housing and touching a wire to one tiny pin on the FPGA chip while the console is on. Go slowly and keep the wire on that one pin. If something looks wrong at any step, stop and read the "If not" notes on that page.

Know the board

The board sits inside the plug housing at the console end of the cable. Open the housing carefully so you can reach both sides of the board, and keep track of any parts you take off.



FPGA side. (1) FPGA U1. (2) Pin 1, by the small triangle. (3) Pin 100 (PROG_B), the leftmost pin of the top row in this view. The J1 pads are labeled on the board: +3.3V, GND, MOSI, CLK, MISO, CS (the square pad).



Other side. (1) DAC U2. (2) Flash U5. (3) IR receiver U7. (4) Button BU1. J1 has the same left-to-right order on this side.

Software setup

Linux (and inside WSL2)

Create a Python virtual environment and install pyftdi. These commands put it in ~/ftdi:

```
python3 -m venv ~/ftdi
~/ftdi/bin/pip install pyftdi
```

Get this repository and go into it:

```
git clone https://github.com/jeff-strutb/carbygcvideo.git
cd carbygcvideo
```

Windows: pass the FT232H to WSL2

Install WSL2 (with Ubuntu) and usbipd-win, for example with `winget install usbipd`. Plug in the FT232H, then open PowerShell **as administrator**:

```
usbipd list
usbipd bind --busid <BUSID>
usbipd attach --wsl --busid <BUSID>
```

Use the BUSID shown next to 0403:6014 in the list. `bind` is needed only the first time. Run `attach` again each time you unplug the FT232H or restart.

Check the FT232H is visible

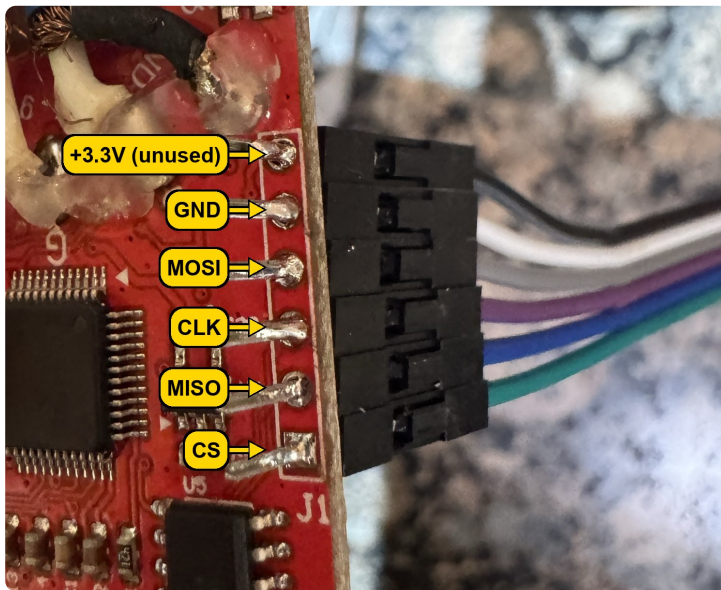
```
lsusb
```

You should see a line with ID 0403:6014 (Future Technology Devices, FT232H).

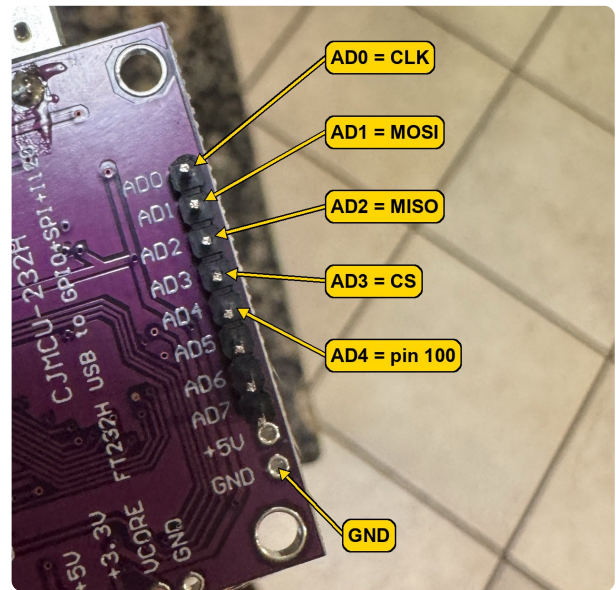
If not: on WSL2, run `sudo apt install usbutils` if `lsusb` is missing, and repeat `usbipd attach`. Try another USB port or cable.

On many systems the scripts need permission to use the USB device. If a script stops with an access or permission error, run it with `sudo` in front, for example `sudo ~/ftdi/bin/python src/flasher/dump.py my_carby_backup.bin`.

Wiring



Header pins soldered to J1, jumper wires attached.



FT232H with a header soldered on AD0 to AD7.

Yes, that is my soldering. It is not pretty, but it worked, and if mine did, yours will too.

Cable J1 pad	FT232H pin
GND	GND
CLK	AD0
MOSI	AD1
MISO	AD2
CS	AD3
FPGA pin 100 (PROG_B), held by hand during flash steps only	AD4

Do not connect +3.3V. The cable is powered by the GameCube, so the console stays on during every step. Feeding 3.3V from the FT232H as well is not needed.

Step 1: back up your firmware

1. Plug the cable into the GameCube and turn the console on.
2. Plug the FT232H into the computer (and attach it to WSL2 on Windows).
3. Run:

```
~/ftdi/bin/python src/flasher/dump.py my_carby_backup.bin
```

4. When it says Hold the AD4 wire tip on pin 100 and keep it there. Press Enter..., touch the AD4 wire to FPGA pin 100, hold it steady, and press Enter.
5. Keep holding until the script finishes (it reads the whole chip twice).

You should see:

```
JEDEC ID: c22013  
Flash size: 4 Mbit  
Saved my_carby_backup.bin sha256: d7e25e1a...
```

Compare your SHA-256 with the original in this repository:

```
cat firmware/original/SHA256SUMS
```

The full value is d7e25e1af843aa424e5c740e0e00690f0429120f4630d0e75bdc3935e503f607. A match means your cable is the same as the one documented here. **Keep your own dump safe either way**, for example copied to another drive.

If not:

- Flash not responding correctly: check GND, CLK, MOSI, MISO and CS, check the console is on, and check the AD4 wire is on pin 100 (not its neighbor).
- The two reads differ or Read was all blank: nothing was saved. Hold the AD4 wire steadier and run it again.

Step 2: flash GCVideo 3.1

1. Same setup as step 1: console on, FT232H wired.

2. Run:

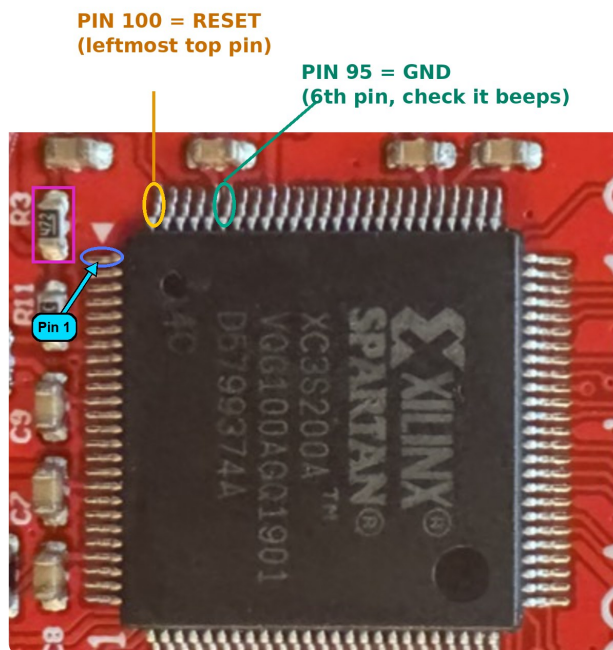
```
~/ftdi/bin/python src/flasher/flashfull.py firmware/gcvideo/  
carby_gcvideo31_full.bin
```

3. It checks the file first (size, update-tool tag, hardware ID CBCG) and prints Complete image: version 3.1-crt, hardware ID CBCG, 346144 bytes. Then it asks: Hold the AD4 wire on pin 100 until DONE. Press Enter...

4. Hold the AD4 wire on pin 100 and press Enter. Keep holding through Erasing... and Writing.... The script then checks every byte it wrote.

5. Let go when it prints DONE – verified OK.

Settings reset to defaults the first time this image starts.



Pin 100 is the leftmost pin of the top row, next to pin 1.

Why hold pin 100? Pin 100 is the FPGA's configuration reset (PROG_B). While it is held low, the FPGA stops running and lets go of the flash chip's wires, so the computer can write to the flash without the FPGA getting in the way.

If not:

- VERIFY FAILED – rerun: do not power-cycle. Run the same command again.
- Flash not responding (ID ...) or Flash write-protected: nothing was written. Check the wiring and the pin 100 contact, then run it again.

Step 3: first boot

1. Turn the GameCube off.
2. Disconnect the FT232H completely, including the AD4 wire. A wire left near pin 100 can reset the FPGA.
3. Connect the TV and turn the console on.

You should see the console's picture with correct colors. When the video mode changes (for example when a game starts), GCVideo shows a small resolution popup in the top right corner.



240p Test Suite color bars.



Color ramps: smooth fades, no steps.

If not:

- Black screen: check that step 2 ended with DONE – verified OK, and flash again. If it still fails, go back to the original (page 10).
- Rolling or diagonal garbage on a CRT: the game or app is outputting 480p. See page 10.

Step 4: set up the remote

The Digital AV port does not carry the controller signal, so the GCVideo menu is opened with an IR remote. Remotes that use the NEC protocol work, which includes many TV and universal remotes.

1. With a picture on screen, hold button BU1 on the board for about a second, until the IR Remote key config screen appears.
2. It asks for six keys in order: Up, Down, Left, Right, Enter, Back. Press the remote key you want for the highlighted one. 0k appears next to it.
3. If Dupe appears, that key is already used. Press a different one.
4. A short press of BU1 cancels without changing anything.

You should see the screen close after the sixth key. Press your Enter key: the GCVideo menu opens.

Keep your changes

In the menu, choose **Store Settings** to save. This also saves the remote keys. Without it, changes are lost at power-off.

Useful settings

- **Advanced Settings, Chroma Interpolation:** turn it off for Game Boy Interface High-Fidelity edition.
- The line doubler is off by default in this build, which is what a standard-definition CRT needs.

If not: if the setup screen never appears, hold BU1 longer. If keys do nothing, the remote may not use the NEC protocol: try another remote.

Going back and troubleshooting

Restore the original firmware

Wire up as for flashing, console on, then:

```
~/ftdi/bin/python src/flasher/restore.py firmware/original/carby_backup.bin
```

Hold the AD4 wire on pin 100, press Enter, and keep holding until DONE – verified OK. You can let go. Then disconnect the FT232H and power-cycle. If it prints VERIFY FAILED, do not power-cycle; run it again.

`restore.py` only writes the original image from this repository (it checks the SHA-256). If your own dump from step 1 differs from it, keep your dump safe and open an issue on the repository before restoring.

Troubleshooting

Symptom	Fix
No change after flashing	Check that the last line was DONE – verified OK. If it was not, rewire CLK and MOSI and flash again.
Flashing seems to work but nothing changes	The flash chip ships write-protected, and it ignores writes silently. <code>flashfull.py</code> clears the protection first. Make sure that is the script you are using.
Screen flashes and resets during commands	The AD4 wire is still touching pin 100. Remove it when not flashing.
Diagonal or rolling garbage on a CRT	The game is outputting 480p, which a standard-definition CRT cannot show. Use 240p or 480i.
Strong flicker in Game Boy Player	That is 480i interlacing. Launch Game Boy Player in 240p from Swiss.